SCIENCEUTSAV®
Science is awesome



Color Mixing and Interactive RGB Systems

Session 3

COLOR MIXING AND INTERACTIVE RGB SYSTEMS

Curiosity Kickoff

- How can such a tiny LED produce bright light?
- What would happen if we connected an LED the wrong way?
- Why are LEDs used in traffic signals instead of normal bulbs?

Concept Connection

This topic introduces students to Digital and Analog Pins through the session Fading Light and Sound. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

1. 1 Watt SMD LED – The Super Torch

I'm flat and powerful. Just one of me can light up your whole table like a torch.

RGB LED

A RGB DIP LED is a single LED with red, green, and blue lights inside; it has one common leg and three color legs, and by turning different color legs ON or OFF, we can create many colors.

BUZZER PCB

A buzzer is an electronic component that generates sound when electric current flows through it. It is commonly utilized in alarms, alerts, and notification systems to provide auditory signals.

Rocker Switch

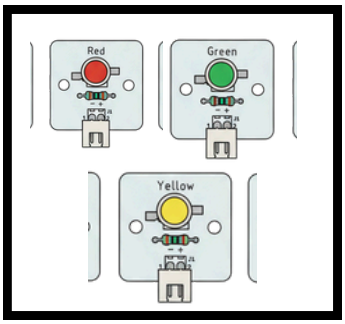
A rocker switch is an electrical device that allows you to turn a circuit ON or OFF by pressing one side of the switch. It is frequently utilized in electronic projects and appliances for convenient power control.

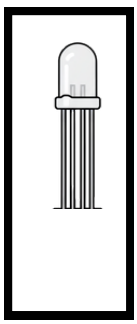
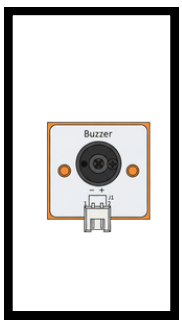
Push Switch

A push switch is a button-style switch that operates only when pressed. It provides temporary input in circuits, such as starting a device or triggering an action.

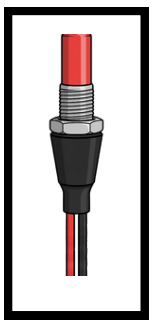
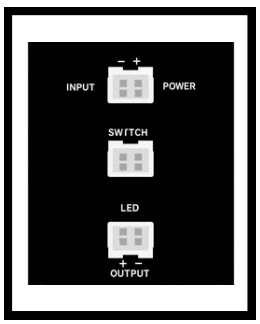
rocker switch PCB

A rocker switch PCB is a small circuit board designed to hold and connect a rocker switch easily in electronic projects.





Beginner: Push Button Basic

Class Explanation

In this class, students learn the basic idea of Push Button Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Yellow 1W SMD LED on breakout board
- 1 × Push Button
- RMC wires
- USB cable

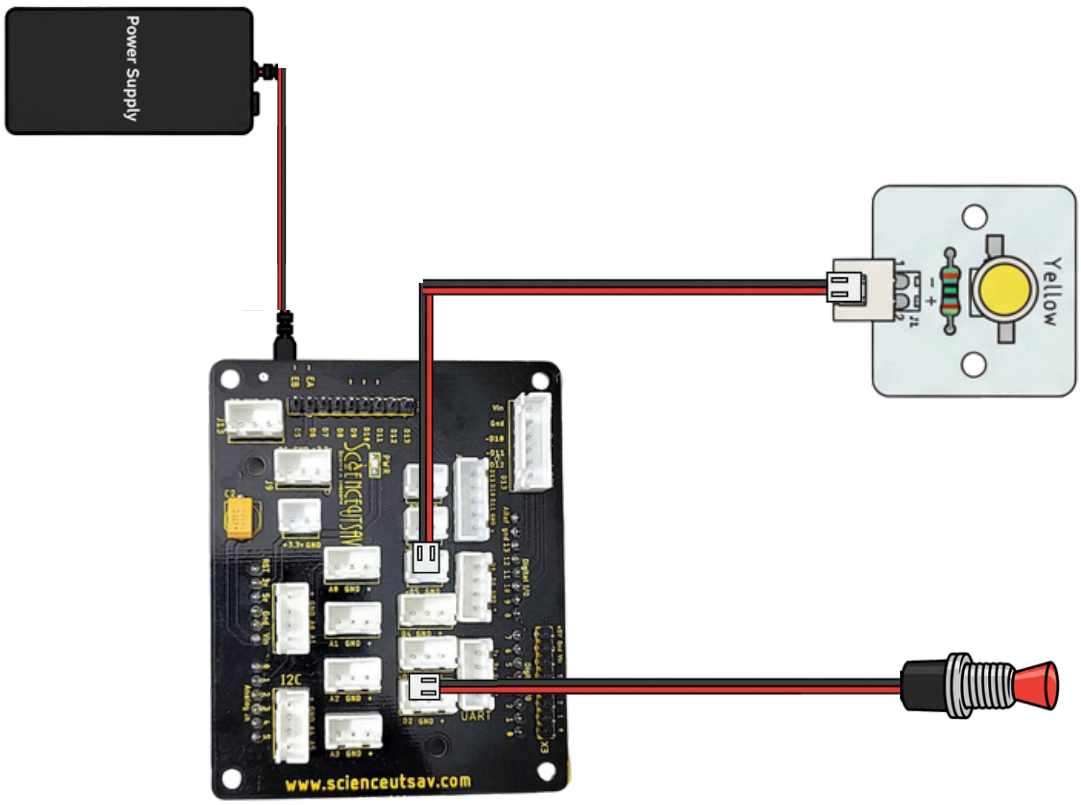
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- Yellow SMD LED → Arduino pin D7 and GND
- Push Button → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int switchPin = 2;
const int ledPin = 5;

void setup() \
 pinMode(switchPin, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 if (digitalRead(switchPin) == LOW) \
 digitalWrite(ledPin, HIGH);
 \ else \
 digitalWrite(ledPin, LOW);
 \
\
```

Code Explanation

- In this Arduino program, a push button is connected to pin 2 (switchPin) and an LED is connected to pin 5 (ledPin).
- The setup() function runs once when the Arduino starts. It sets the push button as INPUT_PULLUP, which keeps the pin HIGH normally and changes it to LOW when the button is pressed. The LED pin is set as OUTPUT.
- The loop() function runs continuously. It reads the button state using digitalRead().
- If the button is pressed (LOW signal), the LED turns ON using digitalWrite(HIGH). If the button is released (HIGH signal), the LED turns OFF using digitalWrite(LOW).
- So overall, the LED glows only while the push button is pressed.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Push Button Basics but changes the pattern, values, or outputs slightly .

Try This Yourself Code

```
const int switchPin = 3;
const int ledPin = 6;

void setup() \
 pinMode(switchPin, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 if (digitalRead(switchPin) == LOW) \
 digitalWrite(ledPin, HIGH);
 \ else \
 digitalWrite(ledPin, LOW);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Push Button Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Push Button with Buzzer

Class Explanation

In this class, students learn the basic idea of Push Button with buzzer. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Buzzer 1W SMD LED on breakout board
- 1 × Push Button
- RMC wires
- USB cable

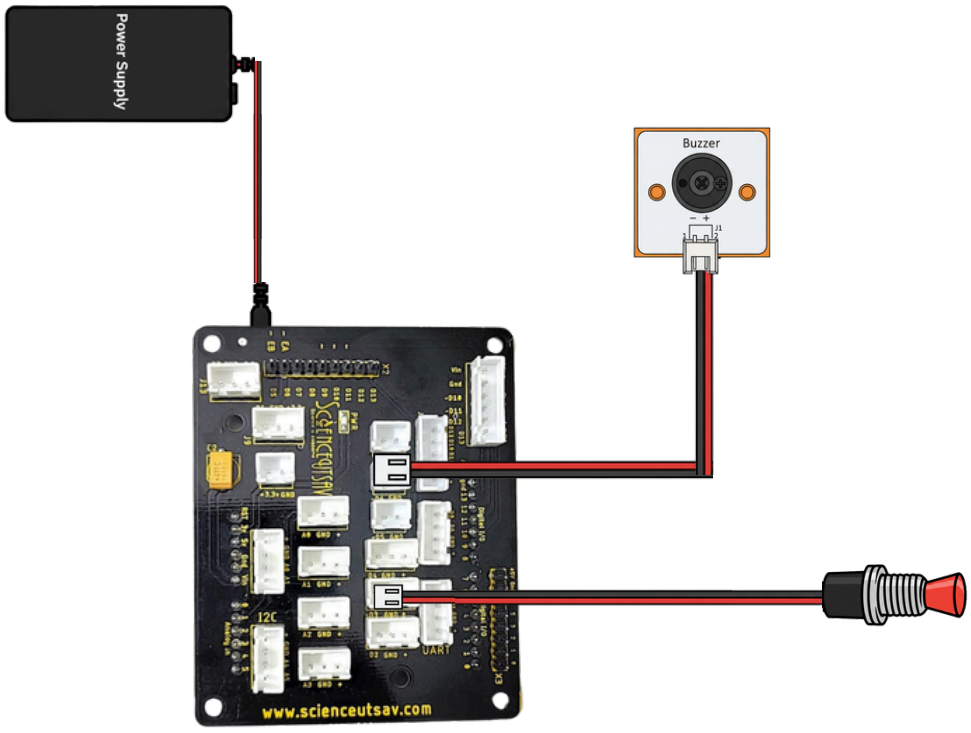
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- 1 × Buzzer 1W SMD LED → Arduino pin D6 and GND
- 1 × Push Button → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int switchPin = 3;
const int buzzerPin = 6;

void setup() \
 pinMode(switchPin, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 if (digitalRead(switchPin) == LOW) \
 digitalWrite(buzzer, HIGH);
 \ else \
 digitalWrite(buzzer, LOW);
 \
\
```

Code Explanation

- In this Arduino program, a push button is connected to pin 3 (switchPin) and a buzzer is connected to pin 6 (buzzerPin).
- The setup() function runs once when the Arduino starts. It sets the push button as INPUT_PULLUP, which keeps the pin HIGH normally and changes it to LOW when the button is pressed. The buzzer pin is set as OUTPUT.
- The loop() function runs continuously. It reads the button state using digitalRead().
- If the button is pressed (LOW signal), the buzzer turns ON using digitalWrite(HIGH). If the button is released (HIGH signal), the buzzer turns OFF using digitalWrite(LOW).
- So overall, the buzzer sounds only while the push button is pressed.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Push Button with buzzer Basics but changes the pattern, values, or outputs slightly .

Try This Yourself Code

```
const int switchPin = 4;
const int buzzerPin = 7;

void setup() \
 pinMode(switchPin, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 if (digitalRead(switchPin) == LOW) \
 digitalWrite(buzzer, HIGH);
 \ else \
 digitalWrite(buzzer, LOW);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Push Button with buzzer how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: RGB control by Push button

Class Explanation

In this class, students learn the basic idea of RGB control by Push button. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × RGB
- 1 × Push Button
- RMC wires
- USB cable

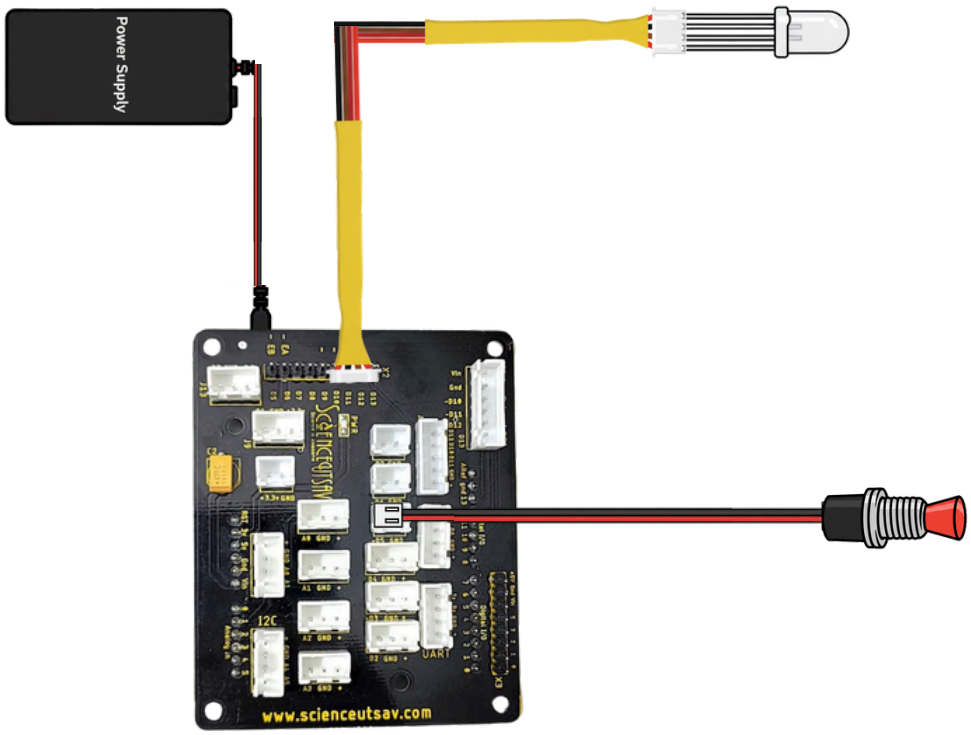
How to Connect

Use RMC wires to connect

- 1 × Push Button → Arduino pin D5 and GND
- RGB → Arduino pins 10 11 12 pins

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int pushSwitch = 5;
const int rgbRed = 10;
const int rgbGreen = 12;
const int rgbBlue = 13;

void setup() {
  pinMode(pushSwitch, INPUT_PULLUP);

  pinMode(rgbRed, OUTPUT);
  pinMode(rgbGreen, OUTPUT);
  pinMode(rgbBlue, OUTPUT);
}

void loop() {
  int switchState = digitalRead(pushSwitch);

  if (switchState == LOW) {
    // Red ON
    digitalWrite(rgbRed, HIGH);
    digitalWrite(rgbGreen, LOW);
    digitalWrite(rgbBlue, LOW);
    delay(500);

    // Green ON
    digitalWrite(rgbRed, LOW);
    digitalWrite(rgbGreen, HIGH);
    digitalWrite(rgbBlue, LOW);
    delay(500);

    // Blue ON
    digitalWrite(rgbRed, LOW);
    digitalWrite(rgbGreen, LOW);
    digitalWrite(rgbBlue, HIGH);
    delay(500);
  } else {
    // Switch not pressed
    digitalWrite(rgbRed, LOW);
    digitalWrite(rgbGreen, LOW);
    digitalWrite(rgbBlue, LOW);
  }
}
```

Code Explanation

- In this Arduino program, a push button is connected to pin 2 (pushSwitch) and an RGB LED is connected to pins 10 (rgbRed), 12 (rgbGreen), and 13 (rgbBlue).
- The setup() function runs once when the Arduino starts. It sets the push button as INPUT_PULLUP and the RGB LED pins as OUTPUT using pinMode().
- The loop() function runs continuously. It reads the push button state using digitalRead() and stores it in switchState.
- If the button is pressed (LOW signal), the RGB LED changes colors in sequence. First, the red LED turns ON for 500 milliseconds, then the green LED turns ON for 500 milliseconds, and finally the blue LED turns ON for 500 milliseconds.
- If the button is not pressed (HIGH signal), all three RGB LED colors remain OFF.
- So overall, pressing the push button makes the RGB LED cycle through red, green, and blue colors, while releasing the button turns the RGB LED OFF.

Try This Yourself Code

This Try This Yourself code keeps the same topic of RGB control by Push button but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int pushSwitch = 2;
const int rgbRed = 10;
const int rgbGreen = 12;
const int rgbBlue = 13;

void setup() {
 pinMode(pushSwitch, INPUT_PULLUP);

 pinMode(rgbRed, OUTPUT);
 pinMode(rgbGreen, OUTPUT);
 pinMode(rgbBlue, OUTPUT);
}

void loop() {
 int switchState = digitalRead(pushSwitch);

 if (switchState == LOW) {
 // Red ON
 digitalWrite(rgbRed, HIGH);
 digitalWrite(rgbGreen, LOW);
 digitalWrite(rgbBlue, LOW);
 delay(500);

 // Green ON
 digitalWrite(rgbRed, LOW);
 digitalWrite(rgbGreen, HIGH);
 digitalWrite(rgbBlue, LOW);
 delay(500);
```

```
 // Blue ON
 digitalWrite(rgbRed, LOW);
 digitalWrite(rgbGreen, LOW);
 digitalWrite(rgbBlue, HIGH);
 delay(500);
 } else {
 // Switch not pressed
 digitalWrite(rgbRed, LOW);
 digitalWrite(rgbGreen, LOW);
 digitalWrite(rgbBlue, LOW);
 }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of RGB control by Push button how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Combining LED Dimmer and Buzzer Tones

Class Explanation

In this class, students learn the basic idea of Combining LED Dimmer and Buzzer Tones. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Red 1W SMD LED on breakout board
- 1 × Buzzer
- RMC wires
- USB cable

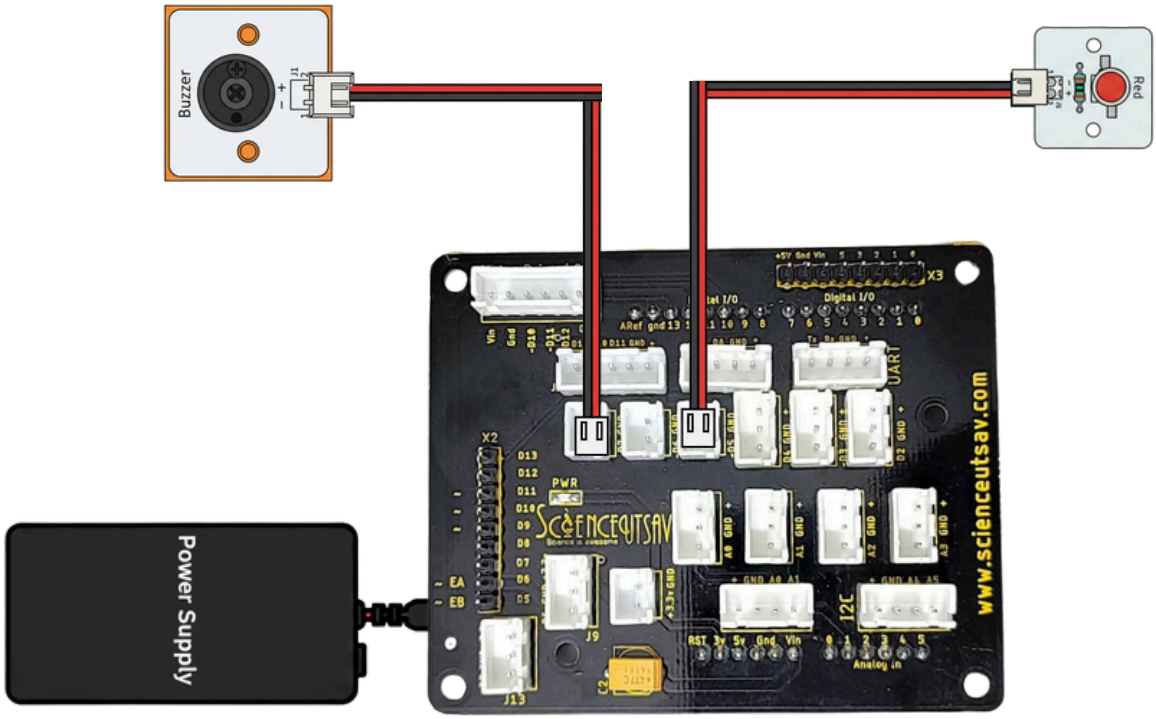
How to Connect

Use RMC wires to connect 1 × Buzzer and 1 × Red 1W SMD LED directly to the Arduino:

- 1 × Red 1W SMD LED → Arduino pin D5 and GND
- 1 × Buzzer → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int ledPin = 5;
const int buzzerPin = 7;

void setup() \
 pinMode(ledPin, OUTPUT);
\

void loop() \
 analogWrite(ledPin, 80);
 tone(buzzerPin, 300);
 delay(700);
 analogWrite(ledPin, 180);
 tone(buzzerPin, 600);
 delay(700);
 analogWrite(ledPin, 255);
 tone(buzzerPin, 900);
 delay(700);
 noTone(buzzerPin);
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 9; or int buzzerPin = 8;, it means we are using those pin numbers and giving them simple names.
- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino what each pin will do. If we write pinMode(ledPin, OUTPUT);, it means the pin will send signals (like turning an LED on or off), and if we use INPUT, the pin will read signals (like from a button).
- After that comes the loop() function. This part runs again and again forever. Arduino keeps repeating whatever is written inside loop, so it can continuously control or check the circuit.
- Inside the loop, we can use analogWrite() to control the brightness of an LED or the speed of a motor using PWM. The value ranges from 0 (off) to 255 (full brightness). We can also use tone() to generate sound from a buzzer by giving a frequency, and noTone() to stop the sound.

So overall, the program first sets everything in setup(), then keeps running the loop continuously, controlling devices like LEDs and buzzers using functions such as pinMode(), analogWrite(), tone(), and noTone().

Try This Yourself Code

This Try This Yourself code keeps the same topic of Combining LED Dimmer and Buzzer Tones but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int ledPin = 5;
const int buzzerPin = 7;

void setup() \
 pinMode(ledPin, OUTPUT);
\

void loop() \
 analogWrite(ledPin, 60);
 tone(buzzerPin, 250);
 delay(500);
 analogWrite(ledPin, 150);
 tone(buzzerPin, 500);
 delay(500);
 analogWrite(ledPin, 255);
 tone(buzzerPin, 800);
 delay(500);
 noTone(buzzerPin);
 delay(300);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Combining LED Dimmer and Buzzer Tones how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes

Objective:

After completing this session, students should be able to:

- Understand how a push button works as an input device to control electronic circuits and trigger simple operations.
- Learn how a push button can activate a buzzer to generate sound for alerts and user interaction.
- Understand how push button inputs are used to control RGB LED colors and switch between different lighting modes.
- Learn how PWM controls LED brightness while generating different buzzer tones, demonstrating combined output control in electronic systems.

Try Out Other Things

this session, students should be able to:

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output